Anexo 04 — Entorno RL de agarrar y lanzar (brazo)

Archivo: Scripts/rl/envs/grab_env.py

Para que sirvio:
Entorno Gymnasium DUMGrabEnv. Implementa la maquina de estados FALLING->HELD->THROWN, el curriculum de dificultad (posicion y caida de la bola) y toda la funcion de recompensa de catch + hold + throw. Base de las policies v7e a v16.

```
"""DUMGrabEnv: Gymnasium environment para entrenar la policy "agarrar y tirar"
con el brazo derecho del Pit Droid.

Sigue literalmente el plan en PLAN_GRAB_REWARD.md:
    - State machine FALLING -> HELD -> THROWN (+ LANDED_OK / FAIL terminal).
    - Reward shaping con constantes (B_grab=500, K_h=200, K_s=150, K_d=400, P_fail=800).
    - r_hold_dynamics piecewise (0-2s lineal positivo, 2-3s neutro, 3s+ exp negativo).
    - Connect equality activo/inactivo via model.eq_active.
    - Patch del qvel al detach (copia ee_linvel al freejoint de la bola).
    - Curriculum por fases (1, 2, 3) que cambia gravedad, spawn, terminacion, throw.

Hereda de MujocoEnv (mismo patron que rl_env.py).
"""

from pathlib import Path
import numpy as np
import mujoco
from gymnasium.envs.mujoco import MujocoEnv
from gymnasium.spaces import Box


# Path al XML grab (clone del DUM4.xml + bola + sites + connects).
XML_PATH = str(Path(__file__).resolve().parents[3] / "Cuerpo" / "DUM4_grab.xml")


# ---------- Constantes del reward (v6: simplificacion radical) ----------
# Solo 3 componentes:
#   r_dist   = -lambda_dist * dist(EE, ball)   (negativo, continuo, dense)
#   r_effort = -lambda_effort * sum(tau^2)     (penalty leve)
#   r_grab   = +B_GRAB_SIMPLE one-shot         (sparse, grande)
# Episode termina al grab o al timeout. Sin floor_fail, sin indecision,
# sin throw, sin vel/z matching, sin time decay.

LAMBDA_DIST   = 10.0     # gradiente fuerte para "chase"
LAMBDA_EFFORT = 1e-4
B_GRAB_SIMPLE = 1000.0   # bonus grande al primer grab

# v7d: timeout penalty bumped (era 250, dominado por -10*dist*steps cuando lejos)
P_TIMEOUT_NO_GRAB = 1000.0

# v7d: shaped grab bonus en los ultimos 8cm para dar gradient denso en zona critica.
# r += SHAPED_BONUS_MAG * (1 - dist/SHAPED_BONUS_DIST) cuando dist < SHAPED_BONUS_DIST
SHAPED_BONUS_DIST = 0.08
SHAPED_BONUS_MAG = 5.0   # max +5 pts/step a dist=0; +2.5 a dist=4cm; 0 a 8cm+

# v7d: spawn relativo al site real (legacy PALM_REST_WORLD ya no se usa)
PALM_REST_WORLD = {
    "R": (-0.012, -0.137, 0.147),
    "L": (+0.241, -0.089, 0.138),
}
INIT_BALL_ABOVE_PALM = 0.03   # 3cm encima del site grip_R/L

# v16: roll-back parcial de v15. cap_up baja a 35cm (era 50 en v15, 40 en v14).
# 35cm es feasible cuando hay caida: la bola pasa por zona alcanzable durante
# su trayectoria. cap_out queda en 15cm (extension lateral, era 10 en v14).
# SIN fase static (que causo el catastrophic forgetting de v15) — falling desde step 0.
CURRICULUM_MAX_OUT = 0.15   # m (extension lateral, era 0.10 en v14)
CURRICULUM_MAX_UP  = 0.35   # m (mas conservador, era 0.50 en v15, 0.40 en v14)

# v10: gravedad de caida (era -2.0 en v8, -0.5 en v7e).
# v12: este valor es el DEFAULT. El callback lo overrides per-step via set_falling_gravity.
FALLING_GRAVITY = -1.5      # m/s2 — default (igual que v11)

# v14: REBALANCED para que el policy realmente aprenda a TIRAR LEJOS, no solo agarrar.
# El problema en v13 fue que el baseline "no swing + dejar caer" daba reward parecido al swing.
# Ahora: hold bajo + velocity bonus alto + landing bonus alto + saturacion lejos.
THROW_GRAB_BONUS = 500.0          # sin cambio
THROW_HOLD_PER_STEP = 1.0         # v14: 3.0 -> 1.0 (no hace falta tanto premio por sostener)
THROW_HELD_MAX_S = 2.5            # sin cambio
THROW_LANDING_K = 1500.0          # v14: 600 -> 1500 (2.5x mas)
THROW_LANDING_DIST_SCALE = 1.5    # v14: 0.5 -> 1.5 (saturacion a 3m en vez de 1m)
THROW_LANDING_BACK_PENALTY = -500.0  # v14: -300 -> -500 (mas castigo si va atras)
EPISODE_MAX_TIME_THROW = 8.0      # sin cambio
```

```
 74 |
 75 | # v12/v14: reward per-step durante THROWN
 76 | THROW_STEP_K_FORWARD = 50.0       # v14: 10 -> 50 (5x — premia ball en vuelo forward continuo)
 77 |
 78 | # v13/v14: bonus al release proporcional a velocidad forward del EE
 79 | THROW_RELEASE_VELOCITY_K = 500.0  # v14: 200 -> 500 (2.5x — swing fuerte vale mucho)
 80 |
 81 | # v12: gravedad de caida es ahora un curriculum (era constante FALLING_GRAVITY=-1.5).
 82 | # El callback empuja per-step un valor entre [FALLING_GRAVITY_MIN, FALLING_GRAVITY_MAX].
 83 | FALLING_GRAVITY_MIN = -1.5     # arranque (igual que v11)
 84 | FALLING_GRAVITY_MAX = -6.0     # final (4x mas realista que v11, sin llegar a -9.81)
 85 |
 86 | # v7d: 10% de los episodios fuerzan offset=0 (rehearsal del caso trivial)
 87 | REHEARSAL_PROB = 0.10
 88 |
 89 | # Legacy v7b/v7c (no se usan en v7d, mantenidas por compat con set_curriculum_active)
 90 | CURRICULUM_INCREMENT = 0.0001
 91 | CURRICULUM_OFFSET_MAX = 0.20
 92 | # "Outward" = -Y world (frente del robot; misma direccion que la palma extendida)
 93 | OUTWARD_AXIS_Y_SIGN = -1.0
 94 |
 95 | # v6b lever-close shaping: DESACTIVADO en v7 (ya no aplica; ball + palm casi tocandose)
 96 | LEVER_CLOSE_BONUS = 0.0
 97 | LEVER_CLOSE_DIST  = 0.0
 98 | LEVER_MAX         = 0.02
 99 |
100 | # Grab detector v7: contacto (dist < 3cm) + lever cerrando (lever > GRAB_LEVER_THR)
101 | # Si el usuario realmente quiso "lever abriendo" (literal), flippear logica abajo.
102 | GRAB_DIST_THR = 0.04     # m EE-bola. Inicial=2.89cm (en grab zone con margen 1.1cm),
103 |                          #   la curriculum lo va sacando (1mm/episodio diagonal).
104 | # v7 interpretacion literal "lever abriendo los dedos":
105 | #   grab = contacto + dedos abiertos (lever_q < umbral pequeno).
106 | #   La policy NO tiene que pelear con la dinamica lenta del lever; solo alinear la palma.
107 | GRAB_LEVER_OPEN_THR = 0.003   # rad — dedos "abiertos" (default ~0)
108 | GRAB_BALL_Z_MIN = 0.10        # m por encima del piso
109 |
110 | # Episode termination
111 | PHASE1_MAX_TIME = 5.0
112 | FLOOR_Z_MARGIN = 0.05    # solo para terminar (sin penalty)
113 |
114 | # Constantes legacy mantenidas para compat (no usadas en v6, pero referenciadas)
115 | LAMBDA_SMOOTH = 0.0      # disabled
116 | RELEASE_LEVER_THR = 0.003
117 | RELEASE_LEVER_QVEL_THR = -0.05
118 | THROW_ACCEL_RADIAL_THR = 8.0
119 | HELD_MAX_TIME = 4.0
120 | EPISODE_MAX_TIME = 8.0
121 | FORWARD_AXIS_Y_SIGN = -1.0
122 | THROW_DIST_PTS_PER_M = 0.0
123 | THROW_HEIGHT_PTS_PER_M = 0.0
124 | THROW_BACK_PTS_PER_M = 0.0
125 | B_GRAB = B_GRAB_SIMPLE   # alias
126 | K_H = 0.0
127 | K_S = 0.0
128 | T_MAX_GRAB = 3.0
129 | H0_HOLD = 0.0
130 | GAMMA_HOLD = 0.0
131 | BUMPED_DIST_THR = 0.05
132 | BUMPED_PARTIAL_FACTOR = 0.0
133 | P_FAIL = 0.0
134 | P_FAIL_PHASE1 = 0.0
135 | INDECISION_PENALTY_PER_STEP = 0.0
136 | INDECISION_LEVER_FLOAT_HI = 0.005
137 | INDECISION_TIME_THR = 0.5
138 |
139 | # Action space (5 dims): HipBody + 3 joints del brazo activo + lever del brazo activo.
140 | # El brazo activo (R o L) se randomiza por episodio. El agente recibe un flag side en obs.
141 | ARM_ACTUATORS = {
142 |     "R": ["act_HipBody", "act_RightShoulderArm", "act_RightForearm",
143 |           "act_RightWrist", "act_RightLever_Slider"],
144 |     "L": ["act_HipBody", "act_LeftShoulderArm", "act_LeftForearm",
145 |           "act_LeftWrist", "act_LeftLever_Slider"],
146 | }
147 | ARM_JOINTS = {
148 |     "R": ["HipBody_joint", "RightShoulderArm_joint", "RightForearm_joint",
149 |           "RightWrist_joint", "RightLever_Slider"],
150 |     "L": ["HipBody_joint", "LeftShoulderArm_joint", "LeftForearm_joint",
151 |           "LeftWrist_joint", "LeftLever_Slider"],
152 | }
153 | EE_SITE = {"R": "grip_R", "L": "grip_L"}
154 | EE_BODY = {"R": "RightWrist_link", "L": "WristLeft_link"}
155 | CONNECT_EQ_NAME = {"R": "ee_ball_connect_R", "L": "ee_ball_connect_L"}
156 | BALL_BODY = "yellow_ball"
157 | BALL_JOINT = "ball_free"
158 | BASE_BODY = "Base_link"
```

```
|
| # Spawn de la bola — la bola cae sobre una recta frontal al robot (eje Y world).
| # Centros adelantados ~0.14m respecto al palma-en-reposo para que el rango [-0.13, +0.13]
| # cubra desde "brazo casi recogido" hasta "extension cercana al maximo".
| SPAWN_CENTER_XY = {
|     "R": (-0.012, -0.275),
|     "L": (+0.241, -0.227),
| }
| SPAWN_RANGE_Y = 0.13       # variacion frontal (sobre el eje "adelante" del robot)
| SPAWN_JITTER_X = 0.0       # 1D puro en Fase 1; aumentar despues si se busca robustez lateral
| # Altura del eje (donde "esta" la palma extendida) — usado como referencia.
| PALM_EXT_Z = {"R": 0.147, "L": 0.138}
| # Altura de spawn por fase (gravedad real g=9.81). Mas alto = mas dificil.
| # Tiempo de caida desde Z hasta la palma: sqrt(2*delta_z/9.81). Para 15cm -> 0.17s.
| SPAWN_Z_RANGE = {
|     1: (0.95, 1.15),   # ~0.45s de caida (Fase facil, triplicado del baseline original)
|     2: (1.30, 1.55),   # ~0.55s de caida (moderada)
|     3: (1.70, 2.10),   # ~0.65s de caida (dificil, mas tiempo para anticipar)
| }
|
| # Curriculum DENTRO de Fase 1: progresivo, controlado por callback de training.
| # 1a: bola estatica (g=0), spawneada al lado de la palma -> aprender solo a cerrar dedos.
| # 1b: caida lenta (g=-3), spawn medio -> aprender a anticipar trayectoria.
| # 1c: caida normal Fase 1 (g=-6), spawn alto -> el reto completo.
| SUBPHASES_F1 = [
|     {"name": "1a_static", "gravity": 0.0,  "spawn_z": (0.18, 0.22)},
|     {"name": "1b_slow",   "gravity": -3.0, "spawn_z": (0.55, 0.75)},
|     {"name": "1c_normal", "gravity": -6.0, "spawn_z": (0.95, 1.15)},
| ]
|
| # Estados
| STATE_FALLING = "FALLING"
| STATE_HELD = "HELD"
| STATE_THROWN = "THROWN"
|
|
| class DUMGrabEnv(MujocoEnv):
|     metadata = {
|         "render_modes": ["human", "rgb_array", "depth_array"],
|         "render_fps": 50,
|     }
|
|     def __init__(self, phase=3, render_mode=None, **kwargs):
|         """phase: 1, 2 o 3 (curriculum).
|             1 = bola estatica, sin throw, sin floor_fail.
|             2 = caida lenta g=3.0, floor_fail x0.3, throw con K_d reducido.
|             3 = g=9.81, spawn aleatorio, todos los pesos pleno.
|         """
|         assert phase in (1, 2, 3), f"phase debe ser 1/2/3, recibi {phase}"
|         self.phase = phase
|
|         # Obs (24,): ball_pos_local(3) + ball_vel_local(3) + ball_z_abs(1)
|         #          + EE_pos_local(3) + EE-ball_delta(3) + arm_qpos(5) + arm_qvel(4) + lever_vel(1)
|         #          + side_flag(1)
|         # side_flag: +1.0 si brazo activo R, -1.0 si L. Permite al agente saber que lado controla.
|         observation_space = Box(low=-np.inf, high=np.inf, shape=(24,), dtype=np.float64)
|
|         MujocoEnv.__init__(
|             self,
|             model_path=XML_PATH,
|             frame_skip=4,
|             observation_space=observation_space,
|             render_mode=render_mode,
|             **kwargs,
|         )
|
|         # Resolver ids para AMBOS lados. En reset, _active_* apunta al lado del episodio.
|         def _resolve_side(side):
|             arm_act_ids = [mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_ACTUATOR, n)
|                            for n in ARM_ACTUATORS[side]]
|             arm_joint_ids = [mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, n)
|                              for n in ARM_JOINTS[side]]
|             return {
|                 "act_ids": arm_act_ids,
|                 "joint_ids": arm_joint_ids,
|                 "qpos_adr": np.array([self.model.jnt_qposadr[j] for j in arm_joint_ids]),
|                 "dof_adr":  np.array([self.model.jnt_dofadr[j]  for j in arm_joint_ids]),
|                 "lever_qpos_adr": int(self.model.jnt_qposadr[arm_joint_ids[-1]]),
|                 "lever_dof_adr":  int(self.model.jnt_dofadr[arm_joint_ids[-1]]),
|                 "lever_act_id":   arm_act_ids[-1],
|                 "ee_site_id": mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_SITE, EE_SITE[side]),
|                 "ee_body_id": mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_BODY, EE_BODY[side]),
|                 "connect_eq_id": mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_EQUALITY, CONNECT_EQ_NAME[side]),
|                 "ctrl_lo": np.array([self.model.actuator_ctrlrange[a, 0] for a in arm_act_ids]),
|                 "ctrl_hi": np.array([self.model.actuator_ctrlrange[a, 1] for a in arm_act_ids]),
```

```
            }
        self._side_info = {"R": _resolve_side("R"), "L": _resolve_side("L")}

        # Bola y base (comunes a ambos lados)
        ball_jid = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_JOINT, BALL_JOINT)
        self._ball_qpos_adr = int(self.model.jnt_qposadr[ball_jid])
        self._ball_dof_adr = int(self.model.jnt_dofadr[ball_jid])
        self._ball_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_BODY, BALL_BODY)
        self._base_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_BODY, BASE_BODY)
        # v13: color randomizado per-episodio (cosmetico, no afecta la policy)
        self._ball_geom_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_GEOM, "ball_geom")

        # Side activo (se setea en reset_model)
        self._side = "R"  # default, override en reset

        # Override action space: 5 dims (HipBody + shoulder + forearm + wrist + lever) en [-1,1]
        # mapeados al brazo activo (R o L segun episodio).
        self.action_space = Box(low=-1.0, high=1.0, shape=(5,), dtype=np.float32)

        # Sub-fase del curriculum (relevante solo si phase==1). Default 2 (mas dificil,
        # equivale al comportamiento sin curriculum). El callback de training la pisa.
        self._subphase_idx = 2

        # v7d: caps actuales del curriculum (los empuja el callback per-step).
        # En cada reset se samplean offsets uniformes en [0, max_*] con prob (1-REHEARSAL_PROB),
        # y con prob REHEARSAL_PROB se fuerzan a 0 (rehearsal del caso trivial).
        self._curriculum_max_up = 0.0
        self._curriculum_max_out = 0.0
        # Offsets EFECTIVOS sampleados este episodio (para info dict y debug)
        self._curriculum_up_offset = 0.0
        self._curriculum_out_offset = 0.0
        # v7e: fase de caida ligera (ultimos 3M de 10M). Lo activa el callback.
        self._falling_active = False
        # v9: throw enabled (HELD->THROWN despues de grab, recompensa throw)
        self._throw_enabled = False
        # v12: gravedad de caida actual (el callback la actualiza per-step)
        self._current_falling_gravity = FALLING_GRAVITY
        # Legacy v7b/v7c (ya no se usan, mantenidos por compat)
        self._curriculum_active = False
        self._curriculum_episode_count = 0

        # Setear gravedad
        self._set_phase_gravity()

        # Snapshot del init_qpos/init_qvel base (despues de cargar)
        self._init_qpos_template = self.init_qpos.copy()
        self._init_qvel_template = self.init_qvel.copy()

        # Piso z (asumido en 0)
        self._z_floor = 0.0

        # Estado del episodio (se inicializa en reset_model)
        self.state = STATE_FALLING
        self.was_held = False
        self.was_bumped = False
        self._grab_bonus_consumed = False
        self.t_spawn = 0.0
        self.t_grab = None
        self.t_release = None
        self._sim_t = 0.0
        self._step_count = 0
        self._prev_action = np.zeros(5, dtype=np.float32)
        self._ball_spawn_pos = np.array([0.0, 0.0, 1.0])
        self._indecision_timer = 0.0
        self._grab_height = None
        self._grab_dt_since_spawn = None
        self._ee_acc_prev = np.zeros(3, dtype=np.float64)  # v5: cache para throw detector
        self._intentional_throw = False                     # v5: flag al release

    # ---------- helpers del lado activo ----------

    @property
    def _arm_act_ids(self):    return self._side_info[self._side]["act_ids"]
    @property
    def _arm_joint_ids(self):  return self._side_info[self._side]["joint_ids"]
    @property
    def _arm_qpos_adr(self):   return self._side_info[self._side]["qpos_adr"]
    @property
    def _arm_dof_adr(self):    return self._side_info[self._side]["dof_adr"]
    @property
    def _lever_qpos_adr(self): return self._side_info[self._side]["lever_qpos_adr"]
    @property
    def _lever_dof_adr(self):  return self._side_info[self._side]["lever_dof_adr"]
    @property
    def _lever_act_id(self):   return self._side_info[self._side]["lever_act_id"]
```

```
    @property
    def _ee_site_id(self):     return self._side_info[self._side]["ee_site_id"]
    @property
    def _ee_body_id(self):     return self._side_info[self._side]["ee_body_id"]
    @property
    def _connect_eq_id(self):  return self._side_info[self._side]["connect_eq_id"]
    @property
    def _arm_ctrl_lo(self):    return self._side_info[self._side]["ctrl_lo"]
    @property
    def _arm_ctrl_hi(self):    return self._side_info[self._side]["ctrl_hi"]

    # ---------- helpers de fase ----------

    def set_subphase(self, idx):
        """Curriculum: actualiza la sub-fase de la Fase 1 (0, 1, o 2).
        Llamado por callback del training. Aplica al proximo reset."""
        self._subphase_idx = int(np.clip(idx, 0, len(SUBPHASES_F1) - 1))

    def set_curriculum_active(self, active: bool):
        """v7 legacy (sin uso en v7d). Mantenida por compat."""
        self._curriculum_active = bool(active)

    def set_curriculum_max_offsets(self, max_up: float, max_out: float):
        """v7d: callback empuja per-step los caps superiores de offset up/out.
        El sample del spawn usa uniform[0, max_*]."""
        self._curriculum_max_up = float(max(0.0, max_up))
        self._curriculum_max_out = float(max(0.0, max_out))

    def set_falling_active(self, active: bool):
        """v7e: activa la caida ligera de la bola (gravedad chica). Llamado por callback."""
        self._falling_active = bool(active)

    def set_throw_enabled(self, enabled: bool):
        """v9: activa el ciclo HELD->THROWN despues del grab (en vez de terminate)."""
        self._throw_enabled = bool(enabled)

    def set_falling_gravity(self, g: float):
        """v12: callback empuja per-step la gravedad de caida (curriculum)."""
        self._current_falling_gravity = float(g)

    def _set_phase_gravity(self):
        """v12: usa self._current_falling_gravity (curriculum) en vez del constante."""
        if self.phase == 1 and self._subphase_idx == 0 and self._falling_active:
            self.model.opt.gravity[:] = (0.0, 0.0, self._current_falling_gravity)
        elif self.phase == 1:
            cfg = SUBPHASES_F1[self._subphase_idx]
            self.model.opt.gravity[:] = (0.0, 0.0, cfg["gravity"])
        else:
            self.model.opt.gravity[:] = (0.0, 0.0, -9.81)

    def _sample_spawn_pos(self):
        """Posicion world inicial de la bola.
        v7 (subphase 0 nuevo): justo arriba de la palma + offset diagonal del curriculum.
            Inicio: palm_rest + (0, 0, INIT_BALL_ABOVE_PALM)  -> 3cm encima
            Luego: cada episodio suma 0.001m alternando up/out hasta CURRICULUM_OFFSET_MAX
            Outward = -Y world (forward del robot)
        Subphases 1/2 mantienen logica de caida legacy.
        """
        if self.phase == 1 and self._subphase_idx == 0:
            # v7d: spawn relativo al site real del EE.
            #   - Con prob REHEARSAL_PROB, force offsets=0 (caso trivial).
            #   - Else, uniform[0, max_*] per dimension. Esto evita catastrophic forgetting:
            #     el rollout buffer siempre contiene una mezcla easy+hard.
            if self.np_random.uniform() < REHEARSAL_PROB:
                self._curriculum_up_offset = 0.0
                self._curriculum_out_offset = 0.0
            else:
                self._curriculum_up_offset = float(self.np_random.uniform(0.0, self._curriculum_max_up))
                self._curriculum_out_offset = float(self.np_random.uniform(0.0, self._curriculum_max_out))
            ee = self.data.site_xpos[self._ee_site_id]
            spawn_x = float(ee[0])
            spawn_y = float(ee[1]) + OUTWARD_AXIS_Y_SIGN * self._curriculum_out_offset
            spawn_z = float(ee[2]) + INIT_BALL_ABOVE_PALM + self._curriculum_up_offset
            return np.array([spawn_x, spawn_y, spawn_z])
        # Fallback: logica legacy con jitter sobre eje frontal
        cx, cy = SPAWN_CENTER_XY[self._side]
        if self.phase == 1:
            z_lo, z_hi = SUBPHASES_F1[self._subphase_idx]["spawn_z"]
        else:
            z_lo, z_hi = SPAWN_Z_RANGE[self.phase]
        cz = float(self.np_random.uniform(z_lo, z_hi))
        dy = float(self.np_random.uniform(-SPAWN_RANGE_Y, SPAWN_RANGE_Y))
        dx = float(self.np_random.uniform(-SPAWN_JITTER_X, SPAWN_JITTER_X)) if SPAWN_JITTER_X > 0 else 0.0
        return np.array([cx + dx, cy + dy, cz])
```

```
414 |     # ---------- Reset ----------
415 |
416 |     def reset_model(self):
417 |         qpos = self._init_qpos_template.copy()
418 |         qvel = self._init_qvel_template.copy()
419 |
420 |         # Reaplicar gravedad por si el curriculum cambio la sub-fase desde el reset anterior
421 |         self._set_phase_gravity()
422 |
423 |         # v7d: el sampling de offsets se hace dentro de _sample_spawn_pos (uniform + rehearsal).
424 |         # Los caps `_curriculum_max_*` los actualiza el callback per-step.
425 |
426 |         # === Randomizar el lado activo (R o L) ===
427 |         self._side = "R" if self.np_random.uniform() < 0.5 else "L"
428 |
429 |         # v13: randomizar color de la bola (cosmetico). Componentes RGB en [0.3, 1.0]
430 |         # para evitar colores muy oscuros y mantenerla visible. Alpha = 1.0.
431 |         if self._ball_geom_id >= 0:
432 |             rgb = self.np_random.uniform(0.3, 1.0, size=3)
433 |             self.model.geom_rgba[self._ball_geom_id, 0] = float(rgb[0])
434 |             self.model.geom_rgba[self._ball_geom_id, 1] = float(rgb[1])
435 |             self.model.geom_rgba[self._ball_geom_id, 2] = float(rgb[2])
436 |             self.model.geom_rgba[self._ball_geom_id, 3] = 1.0
437 |
438 |         # Resetear AMBOS connects (inactivos) — solo se activara el del lado correcto al detectar grab
439 |         for s in ("R", "L"):
440 |             eid = self._side_info[s]["connect_eq_id"]
441 |             self.model.eq_active0[eid] = 0
442 |
443 |         # Set state INICIAL (sin bola en posicion final aun) y hacer settling para que el
444 |         # brazo se asiente. Asi podemos leer la posicion REAL del site grip_* despues
445 |         # del settling y spawnear la bola "apenas arriba" de la palma en su lugar real.
446 |         # (Antes usaba constantes hardcoded que estaban desfasadas 7cm del site real.)
447 |         qpos[self._ball_qpos_adr + 3 : self._ball_qpos_adr + 7] = [1.0, 0.0, 0.0, 0.0]
448 |         qvel[self._ball_dof_adr : self._ball_dof_adr + 6] = 0.0
449 |         self.set_state(qpos, qvel)
450 |         mujoco.mj_forward(self.model, self.data)
451 |         if hasattr(self.data, "eq_active"):
452 |             for s in ("R", "L"):
453 |                 self.data.eq_active[self._side_info[s]["connect_eq_id"]] = 0
454 |         self.data.ctrl[:] = 0.0
455 |         for _ in range(5):
456 |             mujoco.mj_step(self.model, self.data)
457 |
458 |         # AHORA leer site_xpos REAL del EE y spawnear bola relative a el.
459 |         self._ball_spawn_pos = self._sample_spawn_pos()
460 |         self.data.qpos[self._ball_qpos_adr : self._ball_qpos_adr + 3] = self._ball_spawn_pos
461 |         self.data.qpos[self._ball_qpos_adr + 3 : self._ball_qpos_adr + 7] = [1.0, 0.0, 0.0, 0.0]
462 |         self.data.qvel[self._ball_dof_adr : self._ball_dof_adr + 6] = 0.0
463 |         mujoco.mj_forward(self.model, self.data)
464 |
465 |         # Estado del episodio
466 |         self.state = STATE_FALLING
467 |         self.was_held = False
468 |         self.was_bumped = False
469 |         self._grab_bonus_consumed = False
470 |         self.t_spawn = 0.0
471 |         self.t_grab = None
472 |         self.t_release = None
473 |         self._sim_t = 0.0
474 |         self._step_count = 0
475 |         self._prev_action = np.zeros(5, dtype=np.float32)
476 |         self._indecision_timer = 0.0
477 |         self._grab_height = None
478 |         self._grab_dt_since_spawn = None
479 |         self._ee_acc_prev = np.zeros(3, dtype=np.float64)
480 |         self._intentional_throw = False
481 |
482 |         return self._get_obs()
483 |
484 |     # ---------- Geometria / obs ----------
485 |
486 |     def _ball_pos_world(self):
487 |         return self.data.qpos[self._ball_qpos_adr : self._ball_qpos_adr + 3].copy()
488 |
489 |     def _ball_linvel_world(self):
490 |         return self.data.qvel[self._ball_dof_adr : self._ball_dof_adr + 3].copy()
491 |
492 |     def _ee_pos_world(self):
493 |         return self.data.site_xpos[self._ee_site_id].copy()
494 |
495 |     def _ee_linvel_world(self):
496 |         """Velocidad lineal del EE body en world (de cvel: 6D = [ang, lin]).
497 |         cvel esta en coords body-centered; usamos objt_velocity para mayor robustez.
498 |         """
```

```
        vel6 = np.zeros(6)
        mujoco.mj_objectVelocity(
            self.model, self.data,
            mujoco.mjtObj.mjOBJ_BODY, self._ee_body_id, vel6, 0,  # 0 = world frame
        )
        # mj_objectVelocity layout: [angular(3), linear(3)] en world.
        return vel6[3:6].copy()

    def _base_pos_world(self):
        return self.data.xpos[self._base_body_id].copy()

    def _base_mat_world(self):
        return self.data.xmat[self._base_body_id].reshape(3, 3).copy()

    def _world_to_base(self, vec_world):
        """Transforma un vector world a frame del Base_link (rotacion solamente)."""
        return self._base_mat_world().T @ vec_world

    def _get_obs(self):
        base_pos = self._base_pos_world()
        ball_pos_w = self._ball_pos_world()
        ball_vel_w = self._ball_linvel_world()
        ee_pos_w = self._ee_pos_world()

        ball_pos_local = self._world_to_base(ball_pos_w - base_pos)
        ball_vel_local = self._world_to_base(ball_vel_w)
        ee_pos_local = self._world_to_base(ee_pos_w - base_pos)
        ee_ball_delta = ball_pos_local - ee_pos_local
        ball_z_abs = float(ball_pos_w[2])

        arm_qpos = self.data.qpos[self._arm_qpos_adr].copy()  # 5 (HipBody+shoulder+forearm+wrist+lever)
        # arm_qvel: 4 (HipBody+shoulder+forearm+wrist) sin el lever (va separado)
        arm_qvel_4 = self.data.qvel[self._arm_dof_adr[:4]].copy()
        lever_vel = float(self.data.qvel[self._lever_dof_adr])
        side_flag = 1.0 if self._side == "R" else -1.0

        return np.concatenate([
            ball_pos_local,       # 3
            ball_vel_local,       # 3
            [ball_z_abs],         # 1
            ee_pos_local,         # 3
            ee_ball_delta,        # 3
            arm_qpos,             # 5
            arm_qvel_4,           # 4
            [lever_vel],          # 1
            [side_flag],          # 1
        ])

    # ---------- Detectores ----------

    def _detect_grab(self):
        """v7 (lectura literal del spec del usuario):
            grab = contacto palma-bola (dist<3cm) + dedos abiertos (lever_q<3mm) + ball_z>10cm.
        Si el usuario en realidad queria 'lever cerrando', cambiar la condicion abajo.
        """
        if self.state != STATE_FALLING:
            return False
        ee = self._ee_pos_world()
        ball = self._ball_pos_world()
        dist = float(np.linalg.norm(ee - ball))
        if dist >= GRAB_DIST_THR:
            return False
        lever = float(self.data.qpos[self._lever_qpos_adr])
        return (lever < GRAB_LEVER_OPEN_THR) and (ball[2] > GRAB_BALL_Z_MIN)

    def _detect_release(self):
        if self.state != STATE_HELD:
            return False
        lever = float(self.data.qpos[self._lever_qpos_adr])
        lever_v = float(self.data.qvel[self._lever_dof_adr])
        return (lever < RELEASE_LEVER_THR) and (lever_v < RELEASE_LEVER_QVEL_THR)

    def _detect_bumped(self):
        """En FALLING, si el EE roza la bola sin agarrar y esta empieza a alejarse."""
        if self.state != STATE_FALLING:
            return False
        ee = self._ee_pos_world()
        ball = self._ball_pos_world()
        delta = ball - ee
        dist = float(np.linalg.norm(delta))
        if dist > BUMPED_DIST_THR:
            return False
        ball_v = self._ball_linvel_world()
        # ball alejandose del EE => bumped
        return float(np.dot(ball_v, delta)) > 0.05
```

```
584 |
585 |     # ---------- Reward components ----------
586 |
587 |     def _r_approach(self, dt):
588 |         """v5d: POSITIVO, lineal saturado, multiplicado por decay temporal (1 - t/T).
589 |         Decay anti-hover: el approach pierde valor a medida que el episodio avanza,
590 |         forzando al agente a cerrar el grab pronto en vez de quedarse cerca por reward."""
591 |         ee = self._ee_pos_world()
592 |         ball = self._ball_pos_world()
593 |         dist = float(np.linalg.norm(ee - ball))
594 |         d = float(np.clip(dist, 0.0, APPROACH_DIST_FAR)) / APPROACH_DIST_FAR  # 0..1
595 |         val_per_sec = APPROACH_VAL_NEAR + (APPROACH_VAL_FAR - APPROACH_VAL_NEAR) * d
596 |         decay = max(0.0, 1.0 - self._sim_t / APPROACH_DECAY_T)
597 |         return float(val_per_sec * decay * dt)
598 |
599 |     def _compute_ee_accel_world(self):
600 |         """Acel lineal del EE body en world frame via mj_objectAcceleration.
601 |         Llamar despues del ultimo mj_step del frame_skip."""
602 |         mujoco.mj_rnePostConstraint(self.model, self.data)
603 |         acc6 = np.zeros(6)
604 |         mujoco.mj_objectAcceleration(
605 |             self.model, self.data,
606 |             mujoco.mjtObj.mjOBJ_BODY, self._ee_body_id, acc6, 0,
607 |         )
608 |         return acc6[3:6].copy()
609 |
610 |     def _ee_accel_radial_outward(self, ee_acc_world):
611 |         """Proyecta el accel del EE sobre la direccion radial XY hacia afuera del torso."""
612 |         ee = self._ee_pos_world()
613 |         base = self._base_pos_world()
614 |         r_xy = ee[:2] - base[:2]
615 |         n = float(np.linalg.norm(r_xy))
616 |         if n < 1e-6:
617 |             return 0.0
618 |         r_hat = np.array([r_xy[0] / n, r_xy[1] / n, 0.0])
619 |         return float(ee_acc_world @ r_hat)
620 |
621 |
622 |     def _r_hold_dynamics(self, tau):
623 |         """Piecewise:
624 |             tau <= 2: H0 * (1 - tau/2)         # decae lineal de H0 a 0
625 |             2 < tau <= 3: 0                    # neutra
626 |             tau > 3: -gamma * (exp((tau-3)/0.5) - 1)
627 |         """
628 |         if tau <= 2.0:
629 |             return H0_HOLD * (1.0 - tau / 2.0)
630 |         if tau <= 3.0:
631 |             return 0.0
632 |         return -GAMMA_HOLD * (np.exp((tau - 3.0) / 0.5) - 1.0)
633 |
634 |     # ---------- Step ----------
635 |
636 |     def _action_to_arm_ctrl(self, action):
637 |         a = np.clip(action, -1.0, 1.0).astype(np.float64)
638 |         return (a + 1.0) * 0.5 * (self._arm_ctrl_hi - self._arm_ctrl_lo) + self._arm_ctrl_lo
639 |
640 |     def _apply_action(self, action):
641 |         """Construye el vector ctrl completo (otros actuadores fijos en pose neutra
642 |         del XML, que es lo que sale de _action_to_arm_ctrl con [-1,1] mapeado)."""
643 |         full_ctrl = np.zeros(self.model.nu, dtype=np.float64)
644 |         # Otros actuadores: dejar en 0 (posicion = ctrlrange = 0 si el rango incluye 0,
645 |         # o midpoint si no). Vamos a usar midpoint para que sea pose "natural".
646 |         for aid in range(self.model.nu):
647 |             lo, hi = self.model.actuator_ctrlrange[aid]
648 |             # Si 0 esta dentro del rango, usar 0; sino midpoint.
649 |             if lo <= 0.0 <= hi:
650 |                 full_ctrl[aid] = 0.0
651 |             else:
652 |                 full_ctrl[aid] = 0.5 * (lo + hi)
653 |         # Override del brazo derecho con la accion
654 |         arm_ctrl = self._action_to_arm_ctrl(action)
655 |         for i, aid in enumerate(self._arm_act_ids):
656 |             full_ctrl[aid] = arm_ctrl[i]
657 |         return full_ctrl
658 |
659 |     def _patch_ball_velocity_on_release(self):
660 |         """Al desactivar el connect, copiar la velocidad lineal del EE al freejoint
661 |         de la bola (para evitar throw a velocidad cero por bug del solver de equality)."""
662 |         ee_v = self._ee_linvel_world()
663 |         self.data.qvel[self._ball_dof_adr : self._ball_dof_adr + 3] = ee_v
664 |         # qvel angular: dejamos 0 (no nos importa el spin)
665 |         self.data.qvel[self._ball_dof_adr + 3 : self._ball_dof_adr + 6] = 0.0
666 |
667 |     def _set_connect_active(self, active: bool):
668 |         """Activa/desactiva el connect equality en runtime."""
```

```
        val = 1 if active else 0
        # Mujoco usa eq_active0 como el "default" y data.eq_active como el runtime.
        self.model.eq_active0[self._connect_eq_id] = val
        if hasattr(self.data, "eq_active"):
            self.data.eq_active[self._connect_eq_id] = val

    def step(self, action):
        # Ejecutar fisica
        full_ctrl = self._apply_action(action)
        self.do_simulation(full_ctrl, self.frame_skip)
        dt = self.frame_skip * float(self.model.opt.timestep)
        self._sim_t += dt
        self._step_count += 1

        # v9: si estoy en HELD, override del qpos de la bola para que siga la palma.
        # Tambien guardo la velocidad lineal del wrist para usarla al release.
        if self.state == STATE_HELD:
            anchor_world = self._ee_pos_world().copy()
            self.data.qpos[self._ball_qpos_adr : self._ball_qpos_adr + 3] = anchor_world
            # qvel de la bola = velocidad del EE (para suave transicion al release)
            ee_v = self._ee_linvel_world()
            self.data.qvel[self._ball_dof_adr : self._ball_dof_adr + 3] = ee_v
            self.data.qvel[self._ball_dof_adr + 3 : self._ball_dof_adr + 6] = 0.0
            mujoco.mj_forward(self.model, self.data)

        # Calcular accel EE post-fisica. Lo cacheamos *antes* de evaluar release
        # porque al desactivar el connect el qacc se recompone bruscamente.
        ee_acc_world = self._compute_ee_accel_world()

        # === State machine ===
        grabbed_now = False
        released_now = False
        bumped_now = False
        intentional_throw = False

        if self.state == STATE_FALLING:
            if self._detect_grab() and not self._grab_bonus_consumed:
                grabbed_now = True
                self.state = STATE_HELD
                self.t_grab = self._sim_t
                self.was_held = True
                # v9: NO usar self._set_connect_active(True) — la equality connect
                # con freejoint+contype=0 da impulsos numericos enormes en MuJoCo 3.8.
                # En su lugar, override manual del qpos de la bola cada step en HELD
                # (ver mas abajo, post-mj_step).
                anchor_world = self._ee_pos_world().copy()
                self.data.qpos[self._ball_qpos_adr : self._ball_qpos_adr + 3] = anchor_world
                self.data.qvel[self._ball_dof_adr : self._ball_dof_adr + 6] = 0.0
                self._grab_height = float(self._ball_pos_world()[2])
                self._grab_dt_since_spawn = self._sim_t - self.t_spawn
                self._grab_bonus_consumed = True
            elif self._detect_bumped():
                bumped_now = True
                self.was_bumped = True
        elif self.state == STATE_HELD:
            # v9: auto-release tras THROW_HELD_MAX_S segundos en HELD.
            # La bola hereda la velocidad del EE (ya seteada via manual override cada step),
            # asi que al pasar a THROWN sale "tirada" con esa velocidad.
            if self.t_grab is not None and (self._sim_t - self.t_grab) > THROW_HELD_MAX_S:
                released_now = True
                self._intentional_throw = True  # auto-release siempre cuenta como throw
                # v13: cachear velocidad del EE AL MOMENTO del release para el bonus
                self._release_ee_velocity = self._ee_linvel_world().copy()
                self.state = STATE_THROWN
                self.t_release = self._sim_t
                # No _set_connect_active (no usamos equality) ni patch_velocity
                # (ya seteamos qvel cada step en HELD)

        # Guardar accel para el proximo step (usado al detectar release).
        # Solo cacheamos mientras estamos en HELD (la accel sin connect es otra cosa).
        if self.state == STATE_HELD:
            self._ee_acc_prev = ee_acc_world.copy()

        # === Reward v6 simple ===
        ee = self._ee_pos_world()
        ball_pos = self._ball_pos_world()
        dist = float(np.linalg.norm(ee - ball_pos))
        tau_force = self.data.actuator_force[self._arm_act_ids]

        r_dist = -LAMBDA_DIST * dist
        r_effort = -LAMBDA_EFFORT * float(np.sum(tau_force ** 2))

        # v9: si throw enabled, grab bonus reducido (mas peso a hold/throw); si no, simple grab
        grab_bonus_value = THROW_GRAB_BONUS if self._throw_enabled else B_GRAB_SIMPLE
        r_grab_first = grab_bonus_value if grabbed_now else 0.0
```

```
754 |
755 |         # v7d: shaped grab bonus en los ultimos 8cm (gradient denso en zona critica)
756 |         r_lever_close = 0.0
757 |         if dist < SHAPED_BONUS_DIST and self.state == STATE_FALLING:
758 |             r_lever_close = SHAPED_BONUS_MAG * (1.0 - dist / SHAPED_BONUS_DIST)
759 |
760 |         # Variables legacy seteadas en 0 para que info() siga funcionando
761 |         r_smooth = 0.0
762 |         r_approach = 0.0
763 |         r_grab_height = 0.0
764 |         r_grab_speed = 0.0
765 |         r_hold = 0.0
766 |         r_throw_dist = 0.0
767 |         r_throw_height = 0.0
768 |         r_throw_back = 0.0
769 |         r_throw_total = 0.0
770 |         r_floor_fail = 0.0
771 |         r_indecision = 0.0
772 |         r_throw_landing = 0.0  # v9: bonus one-shot al touchdown si throw
773 |         r_throw_velocity = 0.0  # v13: bonus one-shot al release proporcional a vel forward
774 |
775 |         # v9/v11: r_hold per-step mientras HELD (encouragea sostener brevemente).
776 |         # Magnitud: ~3 per step * 125 steps en 2.5s = ~375 total (similar al grab bonus).
777 |         if self._throw_enabled and self.state == STATE_HELD and not grabbed_now:
778 |             r_hold = THROW_HOLD_PER_STEP
779 |
780 |         # v12: r_throw_step (per-step durante THROWN) — bola se aleja forward del robot.
781 |         # forward = -Y world. forward_dist = max(0, ee_y - ball_y) si ball mas adelantada.
782 |         if self._throw_enabled and self.state == STATE_THROWN:
783 |             ee_now = self._ee_pos_world()
784 |             forward_dist = max(0.0, float(ee_now[1] - ball_pos[1]))
785 |             r_throw_dist = THROW_STEP_K_FORWARD * forward_dist * dt
786 |
787 |         # v13: bonus one-shot al release proporcional a vel forward del EE
788 |         if released_now and self._intentional_throw:
789 |             ee_v = getattr(self, "_release_ee_velocity", None)
790 |             if ee_v is not None:
791 |                 forward_v = max(0.0, -float(ee_v[1]))
792 |                 r_throw_velocity = THROW_RELEASE_VELOCITY_K * forward_v
793 |
794 |         # === Terminacion ===
795 |         terminated = False
796 |         truncated = False
797 |         touched_floor = ball_pos[2] < (self._z_floor + FLOOR_Z_MARGIN)
798 |
799 |         if self._throw_enabled:
800 |             # v9 throw flow: grab no termina; touchdown termina y aplica throw reward o penalty
801 |             if touched_floor:
802 |                 terminated = True
803 |                 if self.state == STATE_THROWN:
804 |                     # Bonus por landing: distancia XY desde base, signo positivo si delante
805 |                     base_pos = self._base_pos_world()
806 |                     delta_xy = ball_pos[:2] - base_pos[:2]
807 |                     # forward axis: -Y world (delante del robot)
808 |                     forward = -delta_xy[1]
809 |                     dist_xy = float(np.linalg.norm(delta_xy))
810 |                     if forward > 0.0:
811 |                                         # Tiro hacia adelante: bonus proporcional a la distancia (saturado a
THROW_LANDING_DIST_SCALE)
812 |                         r_throw_landing = THROW_LANDING_K * float(np.tanh(dist_xy / THROW_LANDING_DIST_SCALE))
813 |                     else:
814 |                         # Tiro hacia atras: penalty
815 |                         r_throw_landing = THROW_LANDING_BACK_PENALTY
816 |                 elif self.state == STATE_FALLING:
817 |                     # Bola toco piso sin ser agarrada -> penalty (mismo que timeout sin grab)
818 |                     r_floor_fail = -P_TIMEOUT_NO_GRAB
819 |                 # state == HELD no deberia tocar piso (connect activo)
820 |             elif self._sim_t >= EPISODE_MAX_TIME_THROW:
821 |                 truncated = True
822 |                 if not self.was_held:
823 |                     r_floor_fail = -P_TIMEOUT_NO_GRAB
824 |         else:
825 |             # v7e/v8 flow: termina al grab, sin hold/throw
826 |             if touched_floor:
827 |                 terminated = True
828 |             elif grabbed_now:
829 |                 terminated = True
830 |             elif self._sim_t >= PHASE1_MAX_TIME and self.phase == 1:
831 |                 truncated = True
832 |                 r_floor_fail = -P_TIMEOUT_NO_GRAB
833 |             elif self._sim_t >= EPISODE_MAX_TIME and self.phase != 1:
834 |                 truncated = True
835 |
836 |         reward = (r_dist + r_effort + r_grab_first + r_lever_close + r_floor_fail
837 |                   + r_hold + r_throw_landing + r_throw_dist + r_throw_velocity)
```

```
838 |
839 |         self._prev_action = np.asarray(action, dtype=np.float32).copy()
840 |
841 |         obs = self._get_obs()
842 |         info = {
843 |             "state": self.state,
844 |             "t": self._sim_t,
845 |             "t_grab": self.t_grab,
846 |             "t_release": self.t_release,
847 |             "was_held": self.was_held,
848 |             "was_bumped": self.was_bumped,
849 |             "grab_height": self._grab_height,
850 |             "ball_z": float(ball_pos[2]),
851 |             "ee_ball_dist": float(np.linalg.norm(self._ee_pos_world() - ball_pos)),
852 |             "lever_q": float(self.data.qpos[self._lever_qpos_adr]),
853 |             "r_dist": r_dist,                     # v6 simple
854 |             "r_lever_close": r_lever_close,       # v6b shaping
855 |             "r_smooth": r_smooth, "r_effort": r_effort,
856 |             "r_approach": r_approach, "r_grab_first": r_grab_first,
857 |             "r_grab_height": r_grab_height, "r_grab_speed": r_grab_speed,
858 |             "r_hold": r_hold,
859 |             "r_throw_dist": r_throw_dist, "r_throw_height": r_throw_height,
860 |             "r_throw_back": r_throw_back, "r_throw": r_throw_total,
861 |             "r_floor_fail": r_floor_fail, "r_indecision": r_indecision,
862 |             "grabbed_now": grabbed_now, "released_now": released_now,
863 |             "intentional_throw": self._intentional_throw,
864 |             "phase": self.phase,
865 |         }
866 |         return obs, reward, terminated, truncated, info
867 |
```